



Regression Problem

Linear Regression Model

Thanh-Sach LE

✉ Itsach@hcmut.edu.vn

Faculty of Computer Science and Engineering
Ho Chi Minh City University of Technology

Ho Chi Minh City on February 5, 2026

Linear Regression

Outline

Architecture View

Mathematical
View

Programming
View

Training methods

Outline

- ➊ Architecture View
- ➋ Mathematical View
- ➌ Programming View
- ➍ Training methods



Linear Regression

1 Outline

Architecture View

Mathematical
View

Programming
View

Training methods



Linear Regression

Outline

2 **Architecture View**

Mathematical
View

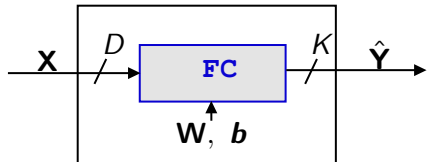
Programming
View

Training methods

Architecture View

Architecture View

Model Architecture



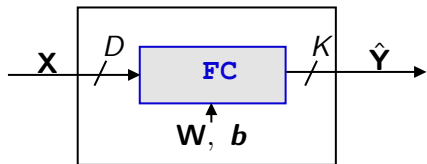
Linear Regression Model

Notations

- **Convention:** vectors are column vectors; batch = stack $\mathbf{x}^T$ as rows; vectors = lowercase bold, matrices = uppercase bold.
- **Input:** $\mathbf{X} \in \mathbb{R}^{B \times D}$ (row n is $\mathbf{x}_n^T$); shape: (B, D)
 - B : batch size; D : input dimension; K : output dimension
- **Labels:** $\mathbf{Y} \in \mathbb{R}^{B \times K}$

Architecture View

Model Architecture



Linear Regression Model

Notations

- **Learnable Parameters: $\mathbf{W}, \mathbf{b}$**
 - $\mathbf{W} \in \mathbb{R}^{D \times K}$; shape: (D, K)
 - $\mathbf{b} \in \mathbb{R}^K$; shape: $(K,)$
 - K : number of outputs
- **Model output: $\hat{\mathbf{Y}} \in \mathbb{R}^{B \times K}$; $\hat{Y}_{n,k}$:**
prediction for sample n , output k



Linear Regression

Outline

4 Architecture View

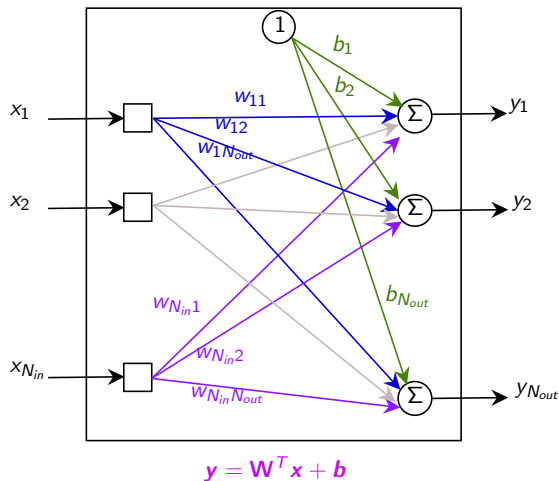
Mathematical View

Programming View

Training methods

Architecture View

Fully-Connected Layer



$$W = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1N_{out}} \\ w_{21} & w_{22} & \cdots & w_{2N_{out}} \\ \vdots & \vdots & \ddots & \vdots \\ w_{N_{in}1} & w_{N_{in}2} & \cdots & w_{N_{in}N_{out}} \end{bmatrix}$$

$$b = [b_1 \quad b_2 \quad \cdots \quad b_{N_{out}}]^T$$

$$x = [x_1 \quad x_2 \quad \cdots \quad x_{N_{in}}]^T$$

$$y = [y_1 \quad y_2 \quad \cdots \quad y_{N_{out}}]^T$$



Linear Regression

Outline

5 Architecture View

Mathematical View

Programming View

Training methods

Architecture View

Fully-Connected Layer



Linear Regression

Outline

6

Architecture View

Mathematical
View

Programming
View

Training methods

Meaning and Math.

- **FC:** Fully-Connected layer
 - **Purpose:** a linear transformation of the input
 - **Formula:**
 - Single sample: $y = W^T x + b$
 - Batch: $\hat{Y} = XW + 1b^T$
 - **Library:**
 - PyTorch: `torch.nn.Linear`
 - Keras: `keras.layers.Dense`



Mathematical View

Linear Regression

Outline

Architecture View

7 Mathematical View

Programming View

Training methods

Mathematical View

Mathematical Formula

Linear Regression

- For a single sample $\mathbf{x} \in \mathbb{R}^D$, the linear regression model computes:

$$\hat{y} = \mathbf{W}^T \mathbf{x} + \mathbf{b}$$

where $\mathbf{W} \in \mathbb{R}^{D \times K}$ is the weight matrix and $\mathbf{b} \in \mathbb{R}^K$ is the bias vector.

- Batch: $\mathbf{X} \in \mathbb{R}^{B \times D}$ (row n is $\mathbf{x}_n^T$):

$$\hat{\mathbf{Y}} = \mathbf{XW} + \mathbf{1b}^T$$

where $\hat{\mathbf{Y}} \in \mathbb{R}^{B \times K}$ is the model output and $\mathbf{b} \in \mathbb{R}^K$ is the bias vector.

Linear Regression is a one-layer neural network without activation.



Linear Regression

Outline

Architecture View

8 Mathematical View

Programming View

Training methods



Programming View

Linear Regression

Outline

Architecture View

Mathematical
View

9 Programming
View

Training methods

Programming View

Code Example



PyTorch: Linear Regression / Linear Classifier

```
import torch
import torch.nn as nn

class LinearModel(nn.Module):
    def __init__(self, input_dim, output_dim):
        super().__init__()
        self.fc = nn.Linear(input_dim, output_dim)

    def forward(self, X): # X: (N, D)
        return self.fc(X)

# Example usage: given D, K
model = LinearModel(input_dim=D, output_dim=K)
```

Linear Regression

Outline

Architecture View

Mathematical
View

10 Programming
View

Training methods

Programming View

Code Example

Keras: Linear Regression / Linear Classifier

```
import tensorflow as tf
from tensorflow.keras import layers, models

class LinearModel(tf.keras.Model):
    def __init__(self, input_dim, output_dim):
        super().__init__()
        self.fc = layers.Dense(output_dim)

    def call(self, X): # X: (N, D)
        return self.fc(X)

# Example usage: given D, K
model = LinearModel(input_dim=D, output_dim=K)

# Build the model (optional but recommended)
model.build(input_shape=(None, D))
```



Linear Regression

Outline

Architecture View

Mathematical
View

11 Programming
View

Training methods



Training methods

Linear Regression

Outline

Architecture View

Mathematical
View

Programming
View

12 Training methods

Training Methods

Overview



Linear Regression

Outline

Architecture View

Mathematical View

Programming View

13

Training methods

Two Ways to Train Linear Regression

- **Closed-form Solution (Analytical)**
 - Solve parameters directly using mathematics
 - No iteration required
 - Applicable only to simple linear models
- **Gradient Descent (Iterative Optimization)**
 - Treat training as an optimization problem
 - Update parameters iteratively
 - General approach for deep learning

Linear Regression is one of the few models that can be trained in both ways.

21

Training Methods

Closed-form Solution

Input, Output and Learnable Parameters

- Input:
 - Batch data: $\mathbf{X} \in \mathbb{R}^{B \times D}$
 - Batch label: $\mathbf{Y} \in \mathbb{R}^{B \times K}$
- Model parameters: $\mathbf{W} \in \mathbb{R}^{D \times K}$, $\mathbf{b} \in \mathbb{R}^K$
- Model output: $\hat{\mathbf{Y}} \in \mathbb{R}^{B \times K}$;

$$\hat{\mathbf{Y}} = \mathbf{X}\mathbf{W} + \mathbf{1}\mathbf{b}^T$$

- Augmented formulation (include bias):

$$\tilde{\mathbf{X}} = [\mathbf{X} \quad \mathbf{1}], \quad \tilde{\mathbf{W}} = \begin{bmatrix} \mathbf{W} \\ \mathbf{b}^T \end{bmatrix}$$



Linear Regression

Outline

Architecture View

Mathematical View

Programming View

14 Training methods

Training Methods

Closed-form Solution



Linear Regression

Outline

Architecture View

Mathematical View

Programming View

15 Training methods

Learning Objective and Analytical Solution

- Objective:

$$\arg \min_{\mathbf{W}, \mathbf{b}} \|\hat{\mathbf{Y}} - \mathbf{Y}\|_F^2$$

- $\|\cdot\|_F$: Frobenius norm (sum of squared errors over all samples)
- Equivalently:

$$\arg \min_{\tilde{\mathbf{W}}} \|\tilde{\mathbf{X}} \tilde{\mathbf{W}} - \mathbf{Y}\|_F^2$$

- Closed-form solution:

$$\tilde{\mathbf{W}} = (\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}})^{-1} \tilde{\mathbf{X}}^\top \mathbf{Y}$$

Training Methods

Closed-form Solution

Closed-form Solution (Ordinary Least Squares)

- Characteristics:
 - Requires matrix inversion
 - Computational cost: $\mathcal{O}(D^3)$
 - Not scalable to large feature dimensions



Linear Regression

Outline

Architecture View

Mathematical View

Programming View

16 Training methods

Training Methods

Closed-form Solution with Regularization

Ridge Regression (L2 Regularization)

- Objective with L2 regularization:

$$\tilde{\mathbf{W}}^* = \arg \min_{\tilde{\mathbf{W}}} \left(\|\tilde{\mathbf{X}} \tilde{\mathbf{W}} - \mathbf{Y}\|_F^2 + \lambda \|\mathbf{W}\|_F^2 \right)$$

- Closed-form solution:

$$\tilde{\mathbf{W}} = \left(\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} + \lambda \begin{bmatrix} \mathbf{I}_D & \mathbf{0} \\ \mathbf{0}^\top & 0 \end{bmatrix} \right)^{-1} \tilde{\mathbf{X}}^\top \mathbf{Y}$$

- Characteristics:
 - Improves numerical stability
 - Controls overfitting
 - Always invertible when $\lambda > 0$
 - Still requires matrix inversion



Linear Regression

Outline

Architecture View

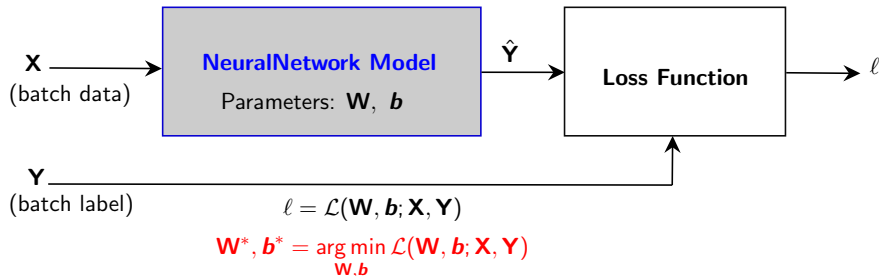
Mathematical View

Programming View

17 Training methods

Training Methods

Gradient Descent: Training Diagram



Linear Regression

Outline

Architecture View

Mathematical View

Programming View

18 Training methods

Training Methods

Gradient Descent



Iterative Optimization

- Treat training as an optimization problem:

$$(\mathbf{W}^*, \mathbf{b}^*) = \arg \min_{\mathbf{W}, \mathbf{b}} \mathcal{L}(\mathbf{W}, \mathbf{b})$$

- Example (squared loss):

$$\mathcal{L}(\mathbf{W}, \mathbf{b}) = \|\hat{\mathbf{Y}} - \mathbf{Y}\|_F^2$$

- Update rules:

$$\mathbf{W}_{t+1} = \mathbf{W}_t - \eta \nabla_{\mathbf{W}} \mathcal{L}$$

$$\mathbf{b}_{t+1} = \mathbf{b}_t - \eta \nabla_{\mathbf{b}} \mathcal{L}$$

Linear Regression

Outline

Architecture View

Mathematical View

Programming View

19 Training methods

Training Methods

Gradient Descent: Notation



Linear Regression

Outline

Architecture View

Mathematical View

Programming View

20 Training methods

Explanation of Mathematical Terms

- $\mathcal{L}(\mathbf{W}, \mathbf{b})$:
 - Loss (objective) function
 - Measures prediction error
- $\nabla_{\mathbf{W}}\mathcal{L}, \nabla_{\mathbf{b}}\mathcal{L}$:
 - Gradients of the loss
 - Direction of steepest increase
- η :
 - Learning rate
 - Step size for parameter updates

Training Methods

Gradient Descent



Linear Regression

Outline

Architecture View

Mathematical View

Programming View

21 Training methods

Iterative Optimization

- Characteristics:
 - Iterative updates
 - No matrix inversion
 - Scales to large datasets
 - Applicable to all neural networks

Gradient Descent is the foundation of deep learning.